

# Reading Code

Martin Kellogg

# Reading code

Today's agenda:

- Why does reading code matter?
- Strategies for reading code effectively
- Role of documentation
- Examples from Covey.Town

# Reading code

Today's agenda:

- Why does reading code
- Strategies for reading code
- Role of documentation
- Examples from Covey.T

## Announcements:

- Individual Project Proposals are due one week from today
- We're finally on a normal schedule next week, with classes on Monday and Wednesday
- Look out for the team formation survey link on Discord
  - due end of next week
- Slides will now be posted ahead of lecture (w/o the reading quiz) due to popular demand

# Reading code

Today's agenda:

- **Why does reading code matter?**
- Strategies for reading code effectively
- Role of documentation
- Examples from Covey.Town

# Why does reading code matter?

- Research suggests that developers spend **up to 70%** [1, 2] of their time trying to understand what code does

[1] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I Know What You Did Last Summer - An Investigation of How Developers Spend Their Time. In Intl. Conf. on Prog. Compr. (ICPC

[2] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring Program Comprehension: A Large-Scale Field Study with Professionals. Trans. on Soft. Eng. (TSE) 44, 10 (2018), 951–976

# Why does reading code matter?

- Research suggests that developers spend **up to 70%** [1, 2] of their time trying to understand what code does
  - This is a **huge** amount of time? Why?

[1] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I Know What You Did Last Summer - An Investigation of How Developers Spend Their Time. In Intl. Conf. on Prog. Compr. (ICPC

[2] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring Program Comprehension: A Large-Scale Field Study with Professionals. Trans. on Soft. Eng. (TSE) 44, 10 (2018), 951–976

# Why does reading code matter?

- Research suggests that developers spend **up to 70%** [1, 2] of their time trying to understand what code does
  - This is a **huge** amount of time? Why?
- Code understanding is a **necessary precursor** for most other development activities:
  - debugging, testing, using an API, adding new features, etc.

[1] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I Know What You Did Last Summer - An Investigation of How Developers Spend Their Time. In Intl. Conf. on Prog. Compr. (ICPC

[2] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring Program Comprehension: A Large-Scale Field Study with Professionals. Trans. on Soft. Eng. (TSE) 44, 10 (2018), 951–976

# Why does reading code matter?

- Research suggests that developers spend **up to 70%** [1, 2] of their time trying to understand what code does
  - This is a **huge** amount of time? Why?
- Code understanding is a **necessary precursor** for most other development activities:
  - debugging, testing, using an API, adding new features, etc.
- Most “code reading” is done **in service to some other goal**
  - i.e., a developer reads code because they want to add a new feature, fix a bug, etc.; not for its own sake

[1] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I Know What You Did Last Summer - An Investigation of How Developers Spend Their Time. In Intl. Conf. on Prog. Compr. (ICPC

[2] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring Program Comprehension: A Large-Scale Field Study with Professionals. Trans. on Soft. Eng. (TSE) 44, 10 (2018), 951–976

# Why does reading code matter?

- Research suggests that developers spend **up to 70%** [1, 2] of their time trying to understand what code does
  - This is a **huge** amount of time? Why?
- Code understanding is a **necessary precursor** for most other development activities:
  - debugging, testing
- Most “code reading” is **not** for its own sake
  - i.e., a developer reads code to understand a new feature, fix a bug, etc.; not for its own sake

**My advice:** Keep the goal in mind whenever you're reading code. It's easy to spend a long time looking at an irrelevant part of the system!

c.

ew

# Auxiliary benefits of reading code

# Auxiliary benefits of reading code

- Reading code can help **build your intuition** for a system
  - makes it easier to diagnose problems, find what you're looking for in the codebase, explain it to others...

# Auxiliary benefits of reading code

- Reading code can help **build your intuition** for a system
  - makes it easier to diagnose problems, find what you're looking for in the codebase, explain it to others...
- Reading code can help you **reduce duplication**
  - “Oh, we don't need to write this—there's already something similar/the same in another file.”

# Auxiliary benefits of reading code

- Reading code can help **build your intuition** for a system
  - makes it easier to diagnose problems, find what you're looking for in the codebase, explain it to others...
- Reading code can help you **reduce duplication**
  - “Oh, we don't need to write this—there's already something similar/the same in another file.”
- Reading code can **make you a better developer**, especially if you're reading good code
  - when writing code, try to emulate the best code you've read!

# Auxiliary benefits of reading code

- Reading code can help **build your intuition** for a system
  - makes it feel like you're already familiar with the code that you're looking at
- Reading code is one of the main advantages of "modern" code review, which we'll discuss later
  - "Oh, we've already seen something like this before. It's already something similar/the same in another file."
- Reading code can **make you a better developer**, especially if you're reading good code
  - when writing code, try to emulate the best code you've read!

**Foreshadowing:** the benefits of reading code are also one of the main advantages of "modern" code review, which we'll discuss later

# Why do we need code reading strategies?

# Why do we need code reading strategies?

- modern software is **unimaginably huge**

# Why do we need code reading strategies?

- modern software is **unimaginably huge**
  - analogy: scale of space vs human scale
    - “Space is big. Really big. You just won't believe how vastly, hugely, mind-bogglingly big it is. I mean, you may think it's a long way down the road to the chemist, but that's just peanuts to space.” – Douglas Adams

# Why do we need code reading strategies?

- modern software is **unimaginably huge**
  - analogy: scale of space vs human scale
    - “Space is big. Really big. You just won't believe how vastly, hugely, mind-bogglingly big it is. I mean, you may think it's a long way down the road to the chemist, but that's just peanuts to space.” – Douglas Adams
  - you will be asked to work on very large software!

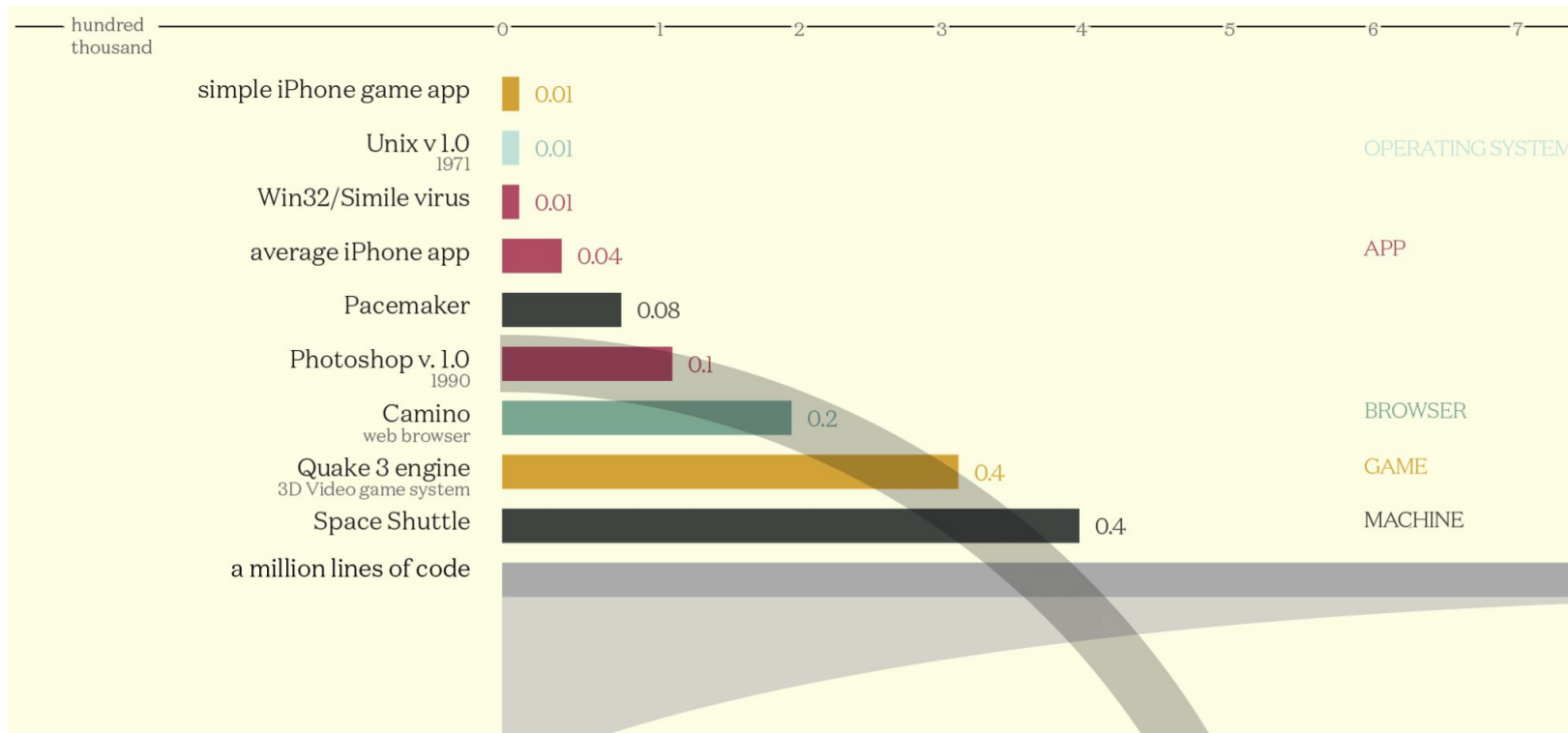
# Why do we need code reading strategies?

- modern software is **unimaginably huge**
  - analogy: scale of space vs human scale
    - “Space is big. Really big. You just won't believe how vastly, hugely, mind-bogglingly big it is. I mean, you may think it's a long way down the road to the chemist, but that's just peanuts to space.” – Douglas Adams
  - you will be asked to work on very large software!
- Techniques developed based on smaller code bases simply **do not apply** or scale to larger code bases

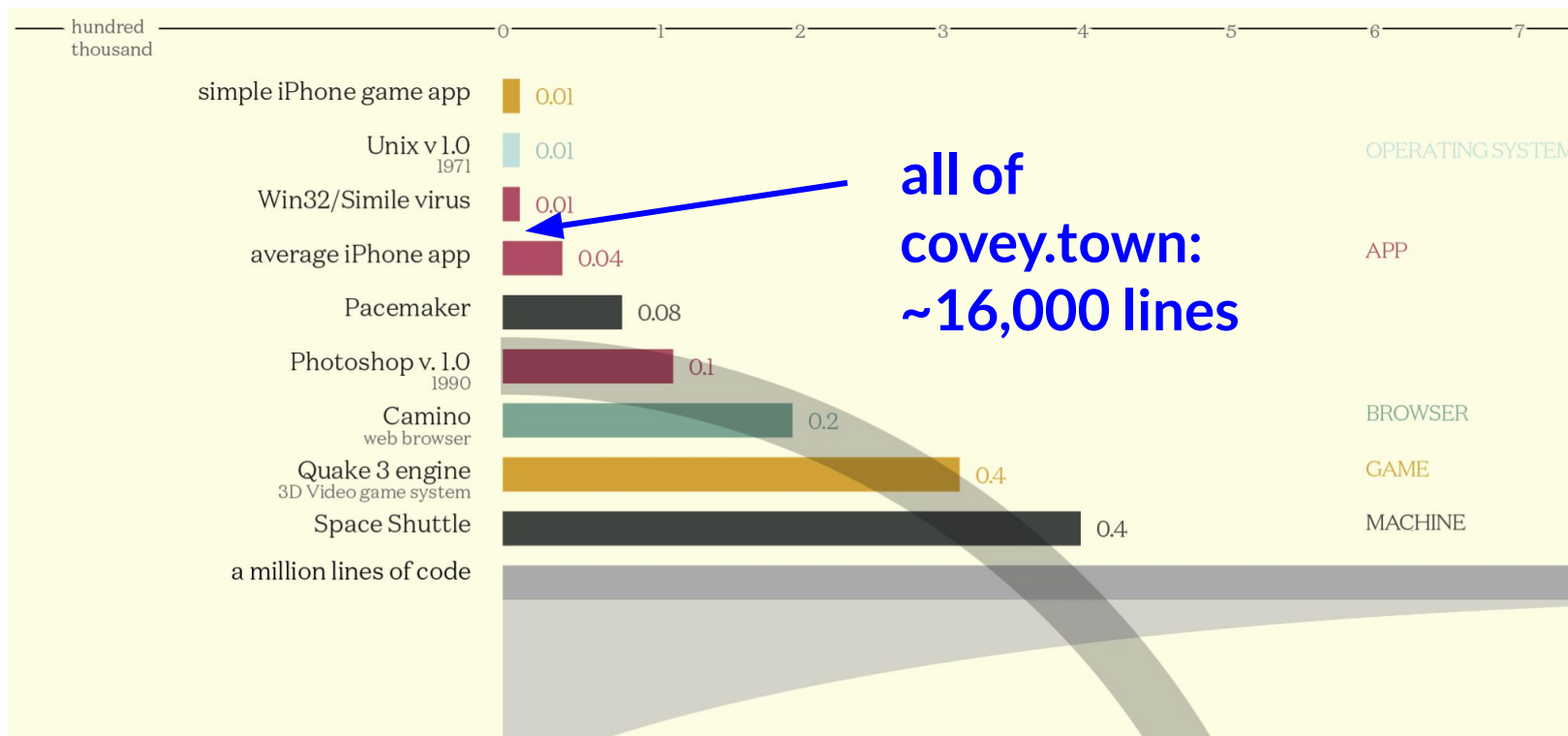
# Why do we need code reading strategies?

- modern software is **unimaginably huge**
  - analogy: scale of space vs human scale
    - “Space is big. Really big. You just won't believe how vastly, hugely, mind-bogglingly big it is. I mean, you may think it's a long way down the road to the chemist, but that's just peanuts to space.” – Douglas Adams
  - you will be asked to work on very large software!
- Techniques developed based on smaller code bases simply **do not apply** or scale to larger code bases
  - Techniques from the 1980s or your habits from classes

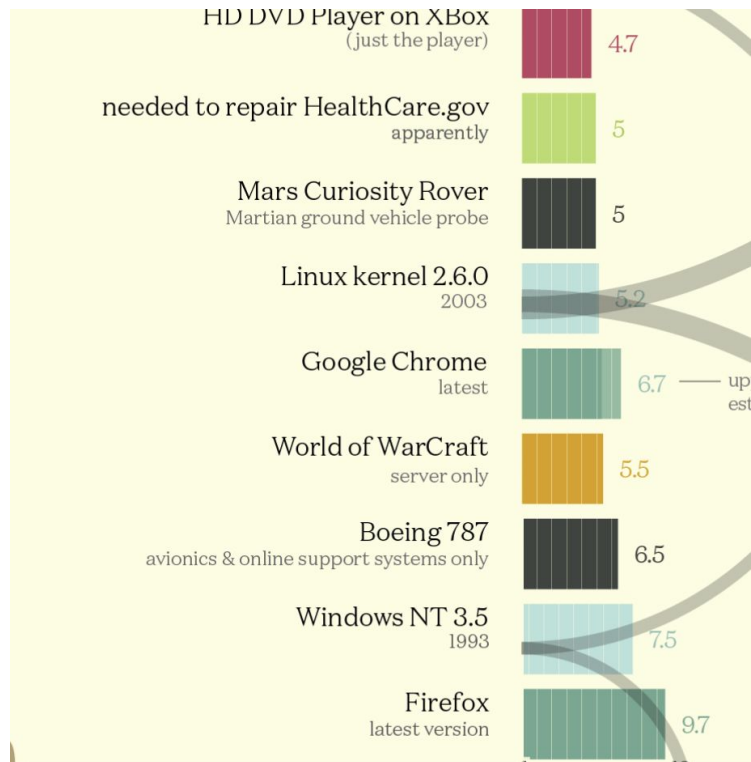
# How big are programs, really?



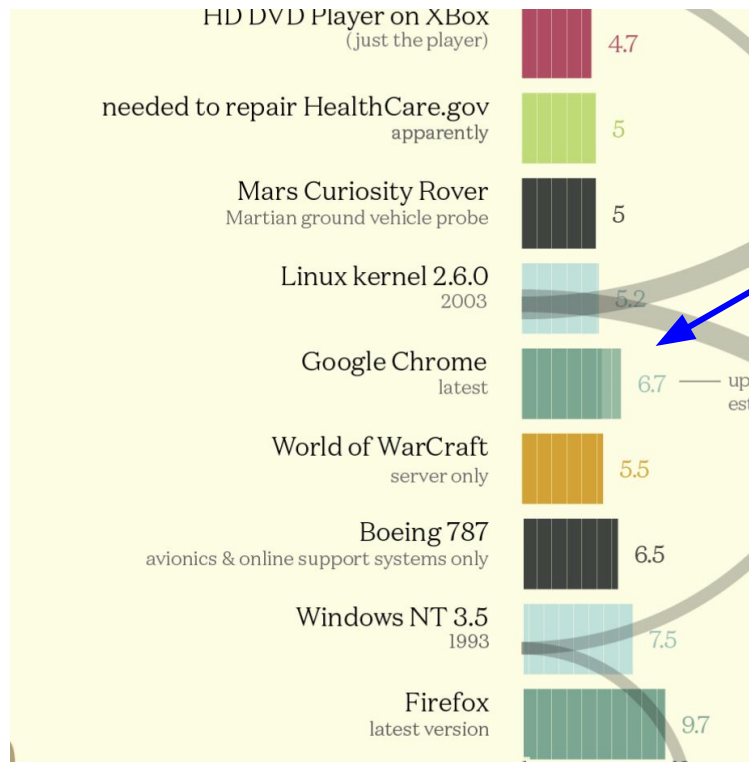
# How big are programs, really?



# How big are programs, really?

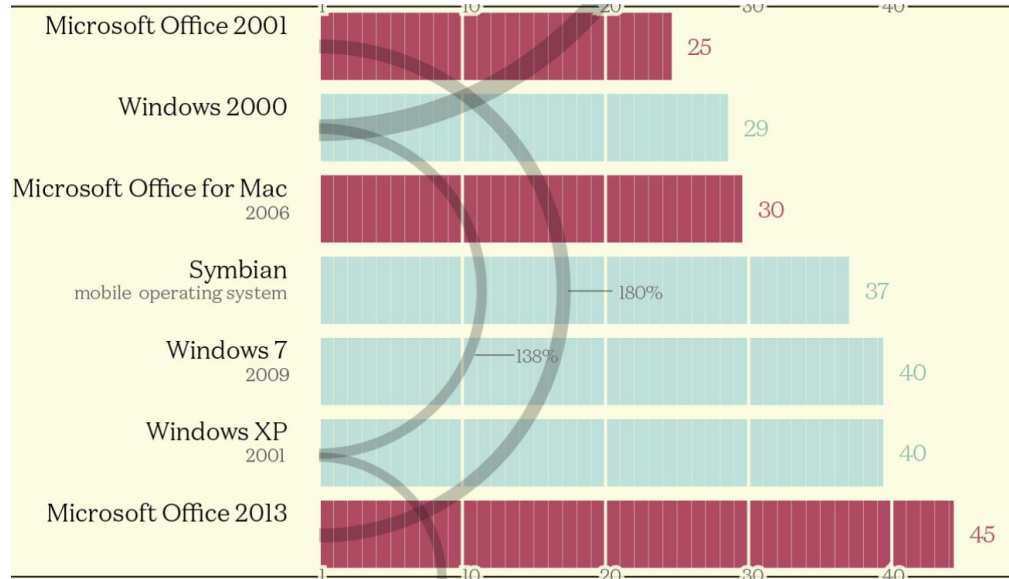
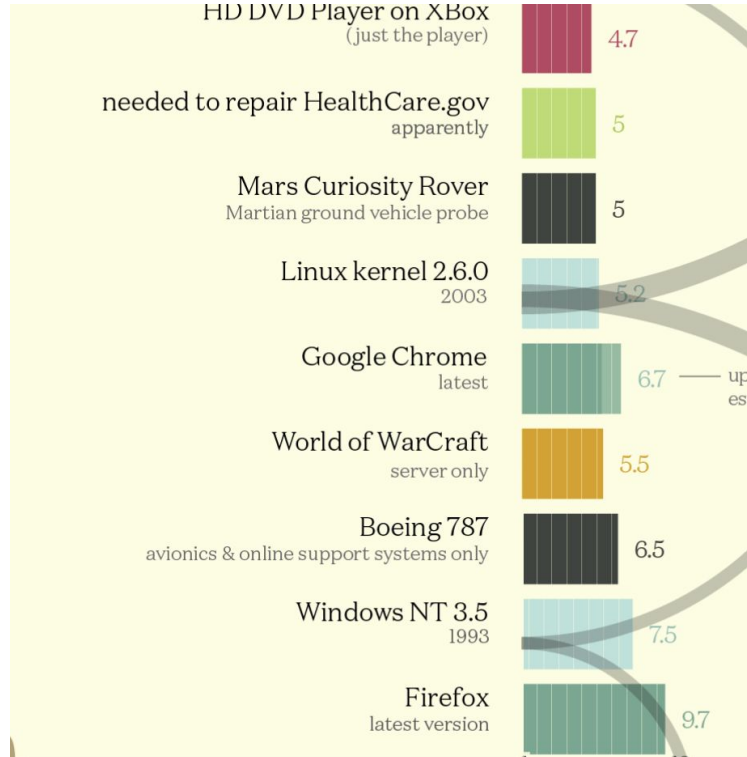


# How big are programs, really?

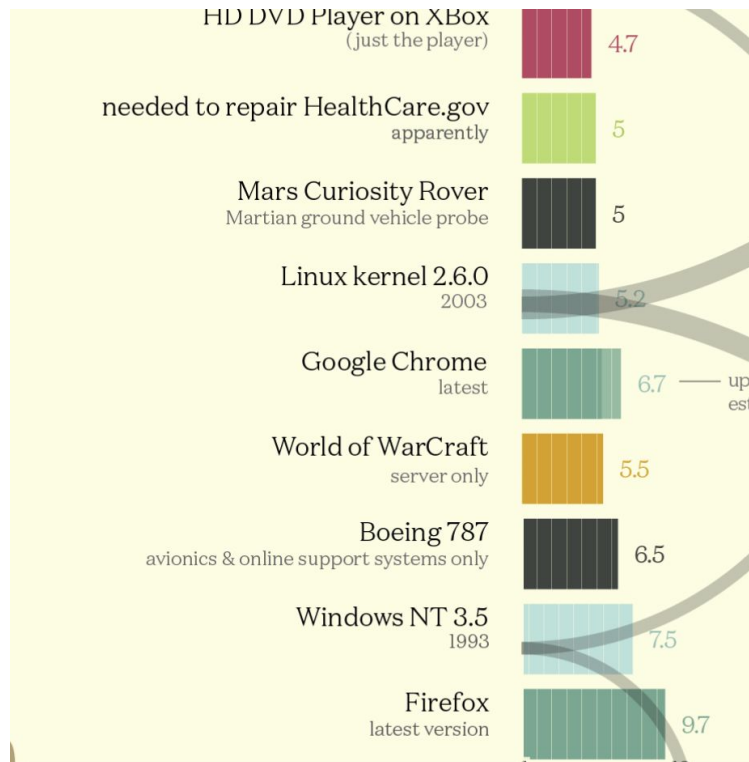


Chrome at ~7M LoC is ~400x bigger than covey.town

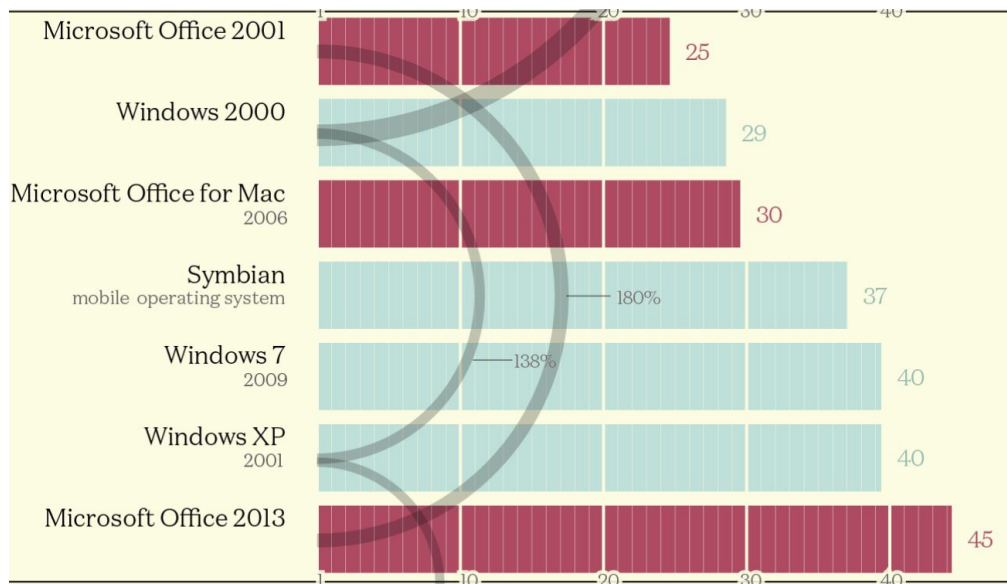
# How big are programs, really?



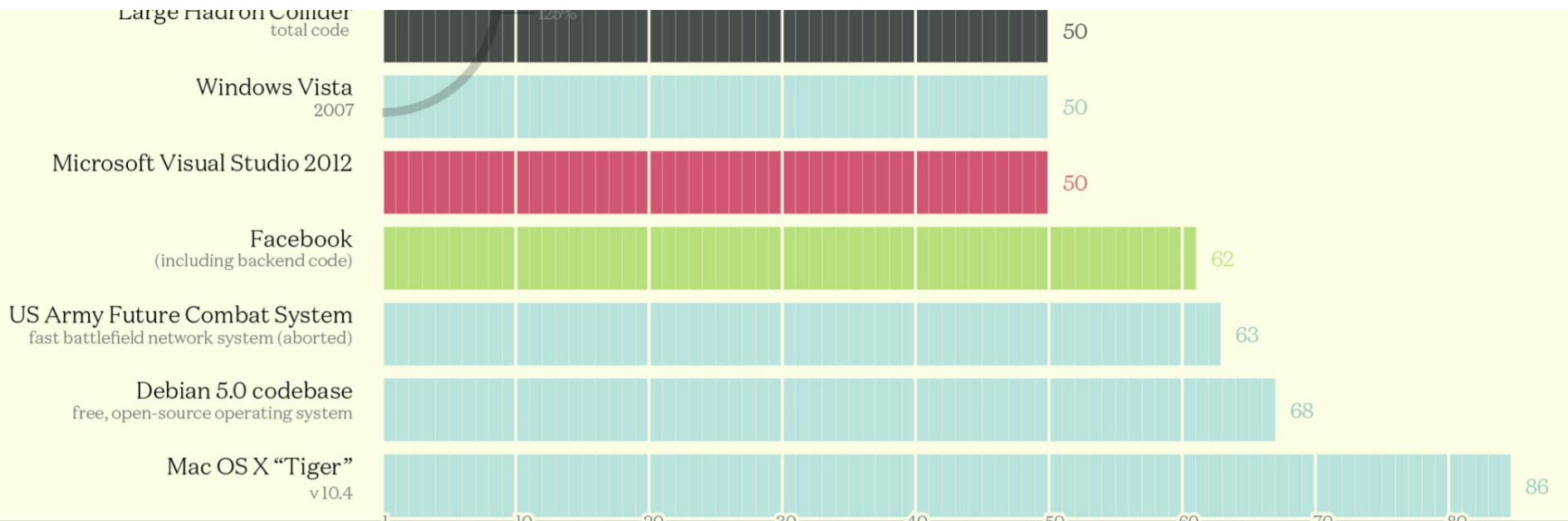
# How big are programs, really?



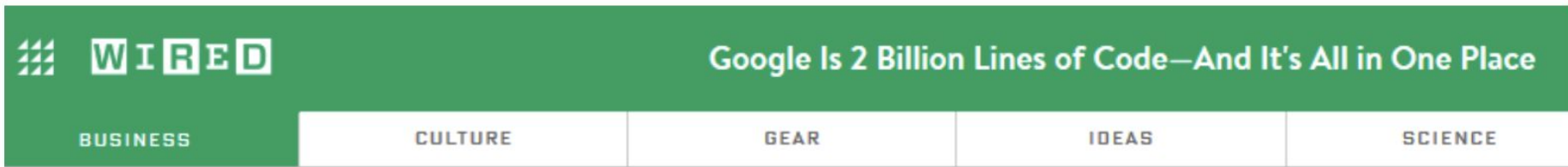
**Chrome is small compared to even old versions of Windows!**



# How big are programs, really?



# How big are programs, really?



CADE METZ BUSINESS 09.16.15 10:00 AM

## GOOGLE IS 2 BILLION LINES OF CODE—AND IT'S ALL IN ONE PLACE

<https://www.wired.com/2015/09/google-2-billion-lines-codeand-one-place/>

# Humans are poor at comprehending large scales

- covey.town 16 000
- google 2 000 000 000

# Humans are poor at comprehending large scales

- covey.town 16 000
- google 2 000 000 000
- Imagine that there is a bug somewhere, **anywhere**, in covey.town

# Humans are poor at comprehending large scales

- covey.town 16 000
- google 2 000 000 000
- Imagine that there is a bug somewhere, **anywhere**, in covey.town
  - Imagine further that you can find that bug in **one minute**

# Humans are poor at comprehending large scales

- covey.town 16 000
- google 2 000 000 000
- Imagine that there is a bug somewhere, **anywhere**, in covey.town
  - Imagine further that you can find that bug in **one minute**
- At the same rate, it would take you **more than a month** to find it in all of google

# Humans are poor at comprehending large scales

- covey.town 16 000
- google 2 000 000 000
- Imagine that there is a bug somewhere, **anywhere**, in covey.town
  - Imagine further that you can find that bug in **one minute**
- At the same rate, it would take you **more than a month** to find it in all of google
  - a one-hour bug on covey.town would take **years** on google!

# Humans are poor at comprehending large scales

- covey.town 16 000

- google 2 000 000 000

- Imagine that It's **not realistic** for you to hold all of the behaviors of such a big system in your head at the same time.

- Imagine

- At the same time

in all of google

- a one-h

# Humans are poor at comprehending large scales

- covey.town 16 000

- google 2 000 000 000

- Imagine that It's **not realistic** for you to hold all of the behaviors of such a big system in your head at the same time.

- Imagine

- At the same time

in all of google

- a one-h

Implication: you need **strategies** for figuring out which parts of the code are actually important to read for the task at hand!

# Strategies for reading code

# Strategies for reading code

- “*Bottom-up*” code comprehension:

# Strategies for reading code

- “*Bottom-up*” code comprehension:
  - start with low-level syntax that you understand

# Strategies for reading code

- “*Bottom-up*” code comprehension:
  - start with low-level syntax that you understand
  - reason through what the code **actually does**

# Strategies for reading code

- “*Bottom-up*” code comprehension:
  - start with low-level syntax that you understand
  - reason through what the code **actually does**
  - build a higher-level abstraction of the code’s **purpose** that you can use to reason about how it’s used

# Strategies for reading code

- “**Bottom-up**” code comprehension:
  - start with low-level syntax that you understand
  - reason through what the code **actually does**
  - build a higher-level abstraction of the code’s **purpose** that you can use to reason about how it’s used
- This technique is best when you have **no preconceived notions** about what the code is for
  - cf. reading a code example on an exam

# Strategies for reading code

- “**Bottom-up**” code comprehension:
  - start with low-level syntax that you understand
  - reason through what the code **actually does**
  - build a higher-level abstraction of the code’s **purpose** that you can use to reason about how it’s used
- This technique is best when you have **no preconceived notions** about what the code is for
  - cf. reading a code example on an exam
- Useful when you’re unfamiliar with the code’s application domain

# Strategies for reading code

- When you're already familiar with an application domain, you can instead use a “**top-down**” comprehension approach:

# Strategies for reading code

- When you're already familiar with an application domain, you can instead use a “**top-down**” comprehension approach:
  - look for familiar structures from the application domain, and scaffold your understanding around them

# Strategies for reading code

- When you're already familiar with an application domain, you can instead use a “**top-down**” comprehension approach:
  - look for familiar structures from the application domain, and scaffold your understanding around them
    - e.g., if you know there must be a database write, you could go looking for that

# Strategies for reading code

- When you're already familiar with an application domain, you can instead use a “**top-down**” comprehension approach:
  - look for familiar structures from the application domain, and scaffold your understanding around them
    - e.g., if you know there must be a database write, you could go looking for that
  - this technique requires you to have some idea of what you're looking for, though

# Strategies for reading code

- In practice, you'll often want something in-between the two extremes of bottom-up and top-down comprehension

# Strategies for reading code

- In practice, you'll often want something in-between the two extremes of bottom-up and top-down comprehension
- Here is one helpful strategy:

# Strategies for reading code

- In practice, you'll often want something in-between the two extremes of bottom-up and top-down comprehension
- Here is one helpful strategy:
  - study what the code's purpose is long enough to **identify one important thing** that it does that you'll recognize
    - e.g., producing output

# Strategies for reading code

- In practice, you'll often want something in-between the two extremes of bottom-up and top-down comprehension
- Here is one helpful strategy:
  - study what the code's purpose is long enough to **identify one important thing** that it does that you'll recognize
    - e.g., producing output
  - then, search the code for that thing and use it as an **anchor**

# Strategies for reading code

- In practice, you'll often want something in-between the two extremes of bottom-up and top-down comprehension
- Here is one helpful strategy:
  - study what the code's purpose is long enough to **identify one important thing** that it does that you'll recognize
    - e.g., producing output
  - then, search the code for that thing and use it as an **anchor**
  - trace the code backwards from there using a bottom-up strategy

# What about documentation?

# What about documentation?

- Most software systems come with some *documentation*: written material describing the functionality and/or purpose of the system
  - e.g., user manuals, design documents, READMEs

# What about documentation?

- Most software systems come with some **documentation**: written material describing the functionality and/or purpose of the system
  - e.g., user manuals, design documents, READMEs
- Documentation is useful for building up a **high-level model** of what a system is supposed to do
  - in particular, you can use it find **anchor** behaviors!

# What about documentation?

- Most software systems come with some **documentation**: written material describing the functionality and/or purpose of the system
  - e.g., user manuals, design documents, READMEs
- Documentation is useful for building up a **high-level model** of what a system is supposed to do
  - in particular, you can use it find **anchor** behaviors!
- However, documentation is limited: it is often **not up to date** vis-a-vis the code (usually because someone forget to update docs)
  - code is the source of truth about what the system actually does

# What about documentation?

- Most software systems come with some **documentation**: written material describing the functionality and/or purpose of the system
  - e.g., user manuals, design documents, READMEs
- Documentation is useful, but it's not always accurate. What a system is supposed to do is often different from what it actually does.
  - in particular, you should be skeptical of documentation that claims to be the source of truth about what the system actually does
- However, documentation is often the only source of information about a system's intended behavior. So, it's important to read it carefully and to compare it with the code.
  - code is the source of truth about what the system actually does

**My advice:** Trust documentation until you see evidence that it's wrong. But, always be willing to dive into the code if there is an inconsistency between docs and the behavior that you observe. **Think critically!**

# “Self-documenting” code

- Some engineers advocate for “*self-documenting*” code—that is, code that follows **naming conventions** and **standard structures** like those we discussed in the last lecture to such an extent that no external documentation is necessary

# “Self-documenting” code

- Some engineers advocate for “*self-documenting*” code—that is, code that follows **naming conventions** and **standard structures** like those we discussed in the last lecture to such an extent that no external documentation is necessary
- Whether this is possible for real systems is still an **open question**

# “Self-documenting” code

- Some engineers advocate for “**self-documenting**” code—that is, code that follows **naming conventions** and **standard structures** like those we discussed in the last lecture to such an extent that no external documentation is necessary
- Whether this is possible for real systems is still an **open question**
- One major criticism of this approach is that self-documenting code might explain *what* it does, but does not explain *why*

# “Self-documenting” code

- Some engineers advocate for “**self-documenting**” code—that is, code that follows **naming conventions** and **standard structures** like those we discussed in the last lecture to such an extent that no external documentation is necessary
- Whether this is possible for real systems is still an **open question**
- One major criticism of this approach is that self-documenting code might explain *what* it does, but does not explain *why*
  - i.e., documentation is necessary to explain the **rationale** for design decisions, what the intended use-case is, etc.

# “Self-documenting” code

- Some engineers advocate for “**self-documenting**” code—that is, code that follows **naming conventions** and **standard structures** like those we discussed in the last lecture to such an extent that no external documentation is needed.
- Whether this is possible is a matter of debate.
- One major criticism is that it gives most of the benefits of self-documenting code anyway. Use documentation to explain **rationale/why**, not what the code does (assume other devs know how to read code, too).
  - i.e., document design decisions, what the intended use-case is, etc.

# Example: how do tile maps work in covey.town?

- Suppose that for a course project, we're interested in making some kind of modification to the “[main map](#)” of covey.town
  - this could be modifying the layout, adding a new area, etc.
- Let's figure out how we would do something like this together!

# Example: how does async/await work?

- Suppose that we want to modify how Town.ts (in the backend) adds a player to the town

# Example: how does async/await work?

- Suppose that we want to modify how Town.ts (in the backend) adds a player to the town
  - it's an "async" function
    - what does that mean?

# async and promises

- Typescript maintains a pool of processes, called *promises*

# async and promises

- Typescript maintains a pool of processes, called *promises*
- A promise always executes until it is completed
  - This is called "*run-to-completion semantics*"

# async and promises

- Typescript maintains a pool of processes, called *promises*
- A promise always executes until it is completed
  - This is called "*run-to-completion semantics*"
- A promise can create other promises to be added to the pool

# async and promises

- Typescript maintains a pool of processes, called *promises*
- A promise always executes until it is completed
  - This is called "*run-to-completion semantics*"
- A promise can create other promises to be added to the pool
- Promises interact mostly by passing values to one another

# async and promises

- Typescript maintains a pool of processes, called *promises*
- A promise always executes until it is completed
  - This is called "*run-to-completion semantics*"
- A promise can create other promises to be added to the pool
- Promises interact mostly by passing values to one another
  - minimizes *data races* (a data race occurs when two instructions from different processes access the same memory location, and at least one of them is a write)

# async and promises

- The reason for the promise system is to mask *latency* (i.e., slow operations) with concurrency

# async and promises

- The reason for the promise system is to mask *latency* (i.e., slow operations) with concurrency
  - Consider: a 1Ghz CPU executes an instruction every 1 ns

# async and promises

- The reason for the promise system is to mask *latency* (i.e., slow operations) with concurrency
  - Consider: a 1Ghz CPU executes an instruction every 1 ns
  - Almost anything else takes forever (approximately)

# async and promises

- The reason for the promise system is to mask *latency* (i.e., slow operations) with concurrency
  - Consider: a 1Ghz CPU executes an instruction every 1 ns
  - Almost anything else takes forever (approximately)
    - e.g., 150,000 ns to read from the SSD, at least 100 million ns just to ping a nearby server over the internet

# async and promises

- The reason for the promise system is to mask *latency* (i.e., slow operations) with concurrency
  - Consider: a 1Ghz CPU executes an instruction every 1 ns
  - Almost anything else takes forever (approximately)
    - e.g., 150,000 ns to read from the SSD, at least 100 million ns just to ping a nearby server over the internet
- Utilize this “wasted” time by *doing something else*

# async and promises

- The reason for the promise system is to mask **latency** (i.e., slow operations) with concurrency
  - Consider: a 1Ghz CPU executes an instruction every 1 ns
  - Almost anything else takes forever (approximately)
    - e.g., 150,000 ns to read from the SSD, at least 100 million ns just to ping a nearby server over the internet
- Utilize this “wasted” time by **doing something else**
  - e.g., processing data, communicating with remote hosts, timers that countdown while our app is running, waiting for users to provide input, etc., by **running a promise**

# async and promises

- The **async** keyword on a function indicates that it creates and returns a promise

# async and promises

- The **async** keyword on a function indicates that it creates and returns a promise
- The **await** keyword means that the current process is **blocked** on some “slow” activity

# async and promises

- The **async** keyword on a function indicates that it creates and returns a promise
- The **await** keyword means that the current process is **blocked** on some “slow” activity
  - allows the runtime (node.js) to move on to some other promise

# async and promises

- The **async** keyword on a function indicates that it creates and returns a promise
- The **await** keyword means that the current process is **blocked** on some “slow” activity
  - allows the runtime (node.js) to move on to some other promise
  - a new promise to return to whatever we were doing is created
    - eligible to run after the “slow” activity finishes

# async and promises

- The **async** keyword on a function indicates that it creates and returns a promise
- The **await** keyword means that the current process is **blocked** on some “slow” activity
  - allows the runtime (node.js) to move on to some other promise
  - a new promise to return to whatever we were doing is created
    - eligible to run after the “slow” activity finishes
- Whenever you do any kind of I/O (or other “slow” activity), you should use the promise system!

# async and promises

- The **async** keyword on a function indicates that it creates and returns a promise
- The **await** keyword means that the current process is **blocked** on some “slow” activity
  - allows the runtime to continue with other tasks
  - a new promise is created
    - eligible to be awaited
- Whenever you do something that might be “slow”, you should use the promise system!

**Aside:** a software engineer can be “blocked” if they’re waiting for something from a coworker. This is a direct analogy to the I/O sense of “blocked” on this slide.

Example: starting a concurrent computation

# Example: starting a concurrent computation

```
async function makeRequest(requestNumber : number) {  
  // some code (to be executed now)  
  const response =  
    await axios.get('https://rest-example.covey.town')  
  // more code (to be executed after the .get() returns).  
}
```

# Example: starting a concurrent computation

```
async function makeRequest(requestNumber : number) {  
  // some code (to be executed now)  
  const response =  
    await axios.get('https://rest-example.covey.town')  
  // more code (to be executed after the .get() returns).  
}
```

- The http request is sent immediately.

# Example: starting a concurrent computation

```
async function makeRequest(requestNumber : number) {  
  // some code (to be executed now)  
  const response =  
    await axios.get('https://rest-example.covey.town')  
  // more code (to be executed after the .get() returns).  
}
```

- The http request is sent immediately.
- A promise is created to run the more code after the http call returns
  - (i.e., the code after “await” is blocked)

# Example: starting a concurrent computation

```
async function makeRequest(requestNumber : number) {  
  // some code (to be executed now)  
  const response =  
    await axios.get('https://rest-example.covey.town')  
  // more code (to be executed after the .get() returns).  
}
```

- The http request is sent immediately.
- A promise is created to run the more code after the http call returns
  - (i.e., the code after “await” is blocked)
- The caller of `makeRequest` resumes immediately.

# General Rules for Writing Asynchronous Code

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.
- Call async procedures only from other async functions or from the top level.

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.
- Call async procedures only from other async functions or from the top level.
- Break up any long-running computation into **async/await** segments so other processes will have a chance to run.

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.
- Call async procedures only from other async functions or from the top level.
- Break up any long-running computation into **async/await** segments so other processes will have a chance to run.
- Leverage concurrency when possible
  - Use **promise.all** if you need to wait for multiple promises to return.

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.
- Consider the following code:

```
async function makeThreeSerialRequests(): Promise<void> {  
    await makeOneGetRequest(1);  
    await makeOneGetRequest(2);  
    await makeOneGetRequest(3);  
    console.log('Heard back from all of the requests')  
}
```
- Use `Promise.all` if you need to wait for multiple promises to return.

# General Rules for Writing Asynchronous Code

- You can't return a value
  - You must send the

“Don't make another request until you got the last response back”

- C
- to
- B
- S
- L

```
async function makeThreeSerialRequests(): Promise<void> {  
    await makeOneGetRequest(1);  
    await makeOneGetRequest(2);  
    await makeOneGetRequest(3);  
    console.log('Heard back from all of the requests')  
}
```

- Use **promise.all** if you need to wait for multiple promises to return.

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.
- ```
async function makeThreeConcurrentRequests() : Promise<void> {  
    await Promise.all([  
        makeOneGetRequest(1),  
        makeOneGetRequest(2),  
        makeOneGetRequest(3)]);  
    console.log('Heard back from all of the requests')  
}
```
- Use **promise.all** if you need to wait for multiple promises to return.

# General Rules for Writing Asynchronous Code

- You can't return a value.
    - You must send the value back to the caller.
  - Call the asynchronous function before you need the result.
    - Use `Promise.all` if you need to wait for multiple promises to return.
- “Make all of the requests now, then wait for all of the responses”
- ```
async function makeThreeConcurrentRequests() : Promise<void> {  
    await Promise.all([  
        makeOneGetRequest(1),  
        makeOneGetRequest(2),  
        makeOneGetRequest(3)]);  
    console.log('Heard back from all of the requests')  
}
```

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.
- Call async procedures only from other async functions or from the top level.
- Break up any long-running computation into **async/await** segments so other processes will have a chance to run.
- Leverage concurrency when possible
  - Use **promise.all** if you need to wait for multiple promises to return.

# General Rules for Writing Asynchronous Code

- You can't return a value from a promise to an ordinary procedure.
  - You must send the value to another promise that is awaiting it.
- Call async procedures only from other async functions or from the top level.
- Break up any long-running computation into **async/await** segments so other processes will have a chance to run.
- Leverage concurrency when possible
  - Use **promise.all** if you need to wait for multiple promises to return.
- Check for errors with **try/catch**

# Takeaways

- Reading code is an important software engineering skill
  - like any skill, it requires **practice!**
- It's usually infeasible to read all of the code, so you should focus on the parts that matter for whatever you're trying to do
- Documentation is often useful, but also often wrong
  - important for context, but for details read the source code
- `async/await` are useful concurrency tools in TypeScript
  - you'll need them for the course project

# Action Items for Next Class

- Start thinking about what feature for Covey.Town you want to propose for your project
  - Everyone will need to come up with a (one-page!) feature proposal by next week (due **one week from today**)
  - But you're not "locked in" to the feature that you choose
- Mandatory readings for Process